

Task A: Dirty Read

In Session 1, start a transaction and update the price of a room without committing.

In Session 2, try to read the updated room price before Session 1 commits or rolls back.

Experiment with READ UNCOMMITTED and READ COMMITTED isolation levels.

SQLQuery2.sql...install (51))*SQLQuery1.sql...n stall (97))*

```
1  USE HotelBookingDB;
2  GO
3
4  BEGIN TRANSACTION;
5
6      -- Update room price WITHOUT committing
7      UPDATE Rooms
8      SET Price = 999.99
9      WHERE RoomID = 1;
10
11      PRINT 'Session 1: Price updated to 999.99 - NOT committed yet. Keep this window open.';
12
13      -- !! Do NOT commit or rollback yet. Switch to Session 2 now. !!
```

100 % 3 0

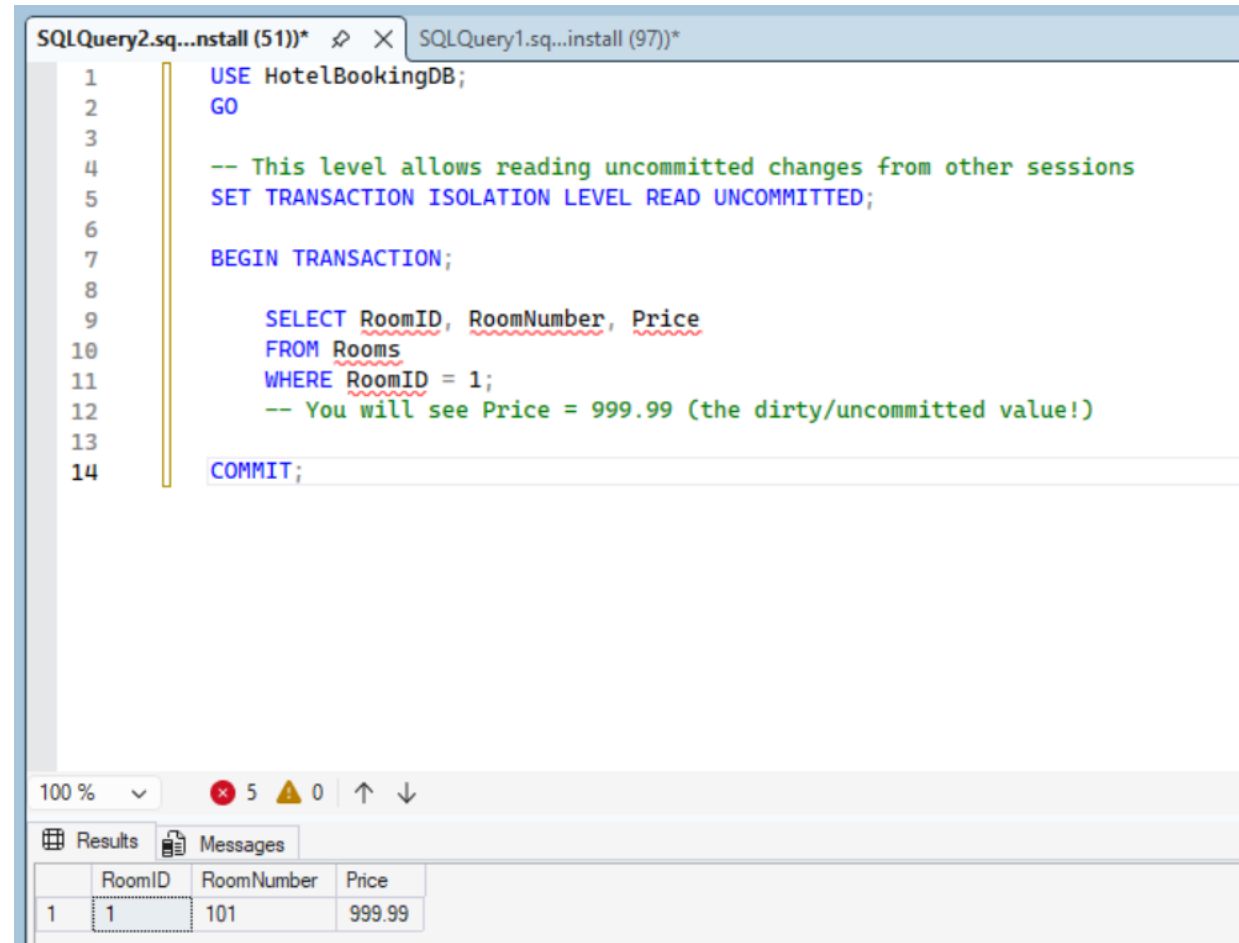
Messages

(1 row affected)
Session 1: Price updated to 999.99 - NOT committed yet. Keep this window open.

Completion time: 2026-02-27T17:46:25.0524985+05:30

The **session 2** read the uncommitted value of price which was updated by the session 1 but was not committed to database.

This leads to the problem of **dirty read**.



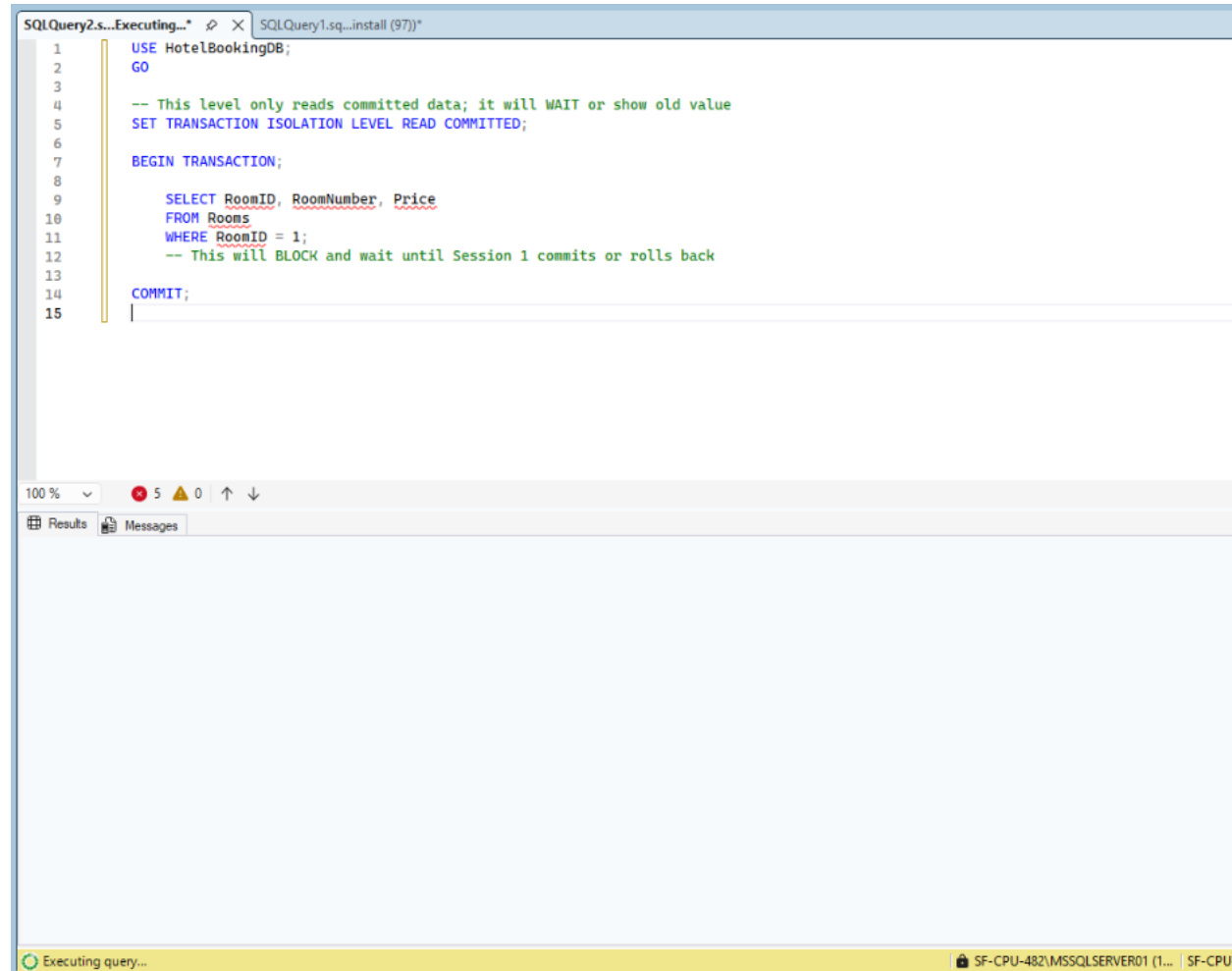
```
SQLQuery2.sql...ninstall (51))*  SQLQuery1.sql...install (97))*
1  USE HotelBookingDB;
2  GO
3
4  -- This level allows reading uncommitted changes from other sessions
5  SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
6
7  BEGIN TRANSACTION;
8
9      SELECT RoomID, RoomNumber, Price
10     FROM Rooms
11    WHERE RoomID = 1;
12    -- You will see Price = 999.99 (the dirty/uncommitted value!)
13
14 COMMIT;
```

100 % 5 0

Results Messages

	RoomID	RoomNumber	Price
1	1	101	999.99

Now we'll run the **session 2** with read committed isolation level and as we can see in the below given image, it waits for **session 1** to release its lock and then it can access the page.

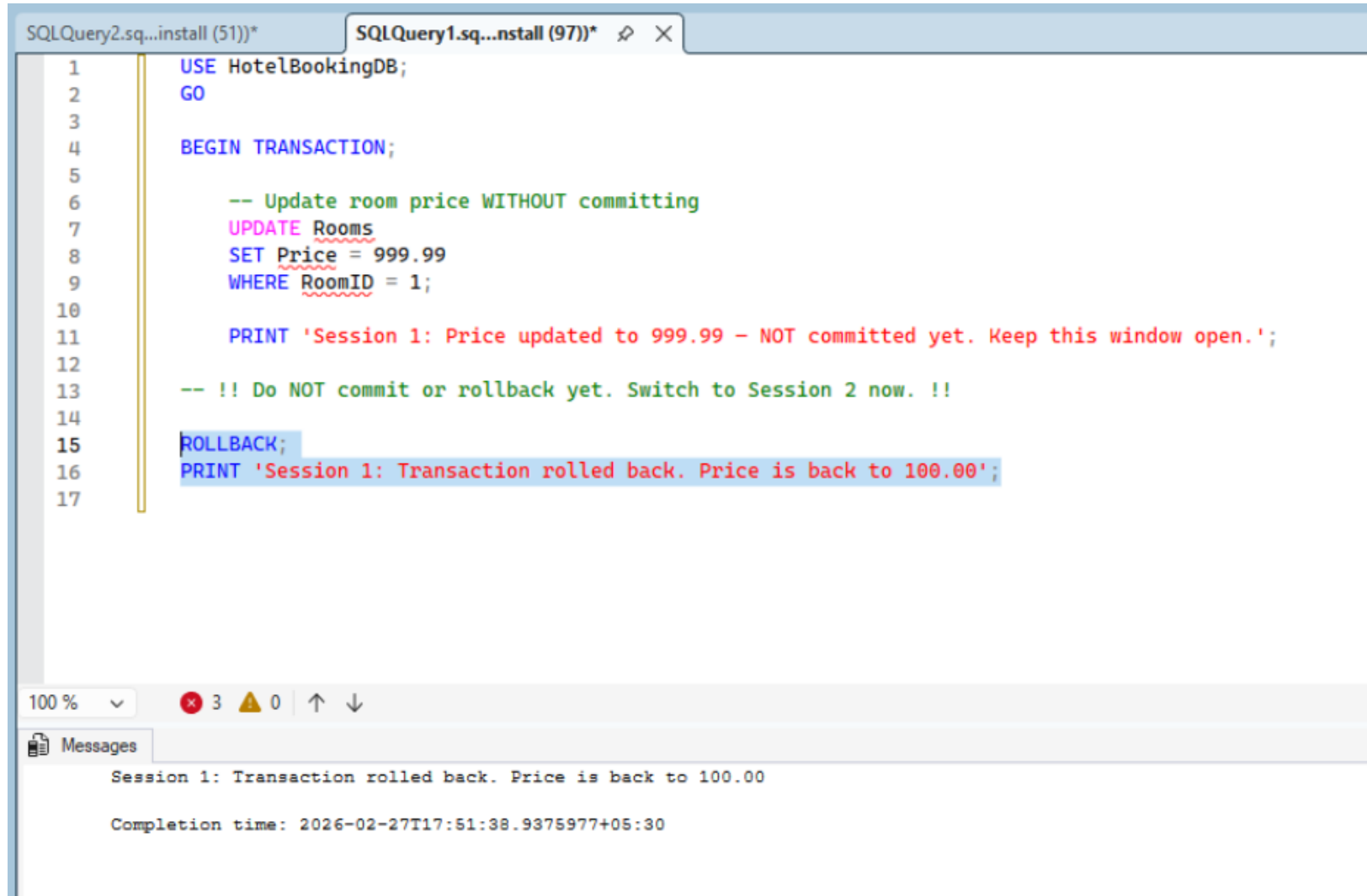


The screenshot shows a SQL Server Enterprise Manager window titled "SQLQuery2.s...Executing...". The query text is as follows:

```
1  USE HotelBookingDB;
2  GO
3
4  -- This level only reads committed data; it will WAIT or show old value
5  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
6
7  BEGIN TRANSACTION;
8
9      SELECT RoomID, RoomNumber, Price
10     FROM Rooms
11     WHERE RoomID = 1;
12     -- This will BLOCK and wait until Session 1 commits or rolls back
13
14 COMMIT;
15
```

The status bar at the bottom indicates "Executing query..." and the server name is "SF-CPU-482\MSSQLSERVER01 (1...)".

Now coming back to **session 1** and rolling back it leads to release of its locks and then the **session 2** can access the page which was being locked by **session 1**.



The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are two tabs: 'SQLQuery2.sql...install (51))*' and 'SQLQuery1.sql...install (97))*'. The active tab is 'SQLQuery1.sql...install (97))*'. The main area displays a SQL script with line numbers 1 through 17. The script is as follows:

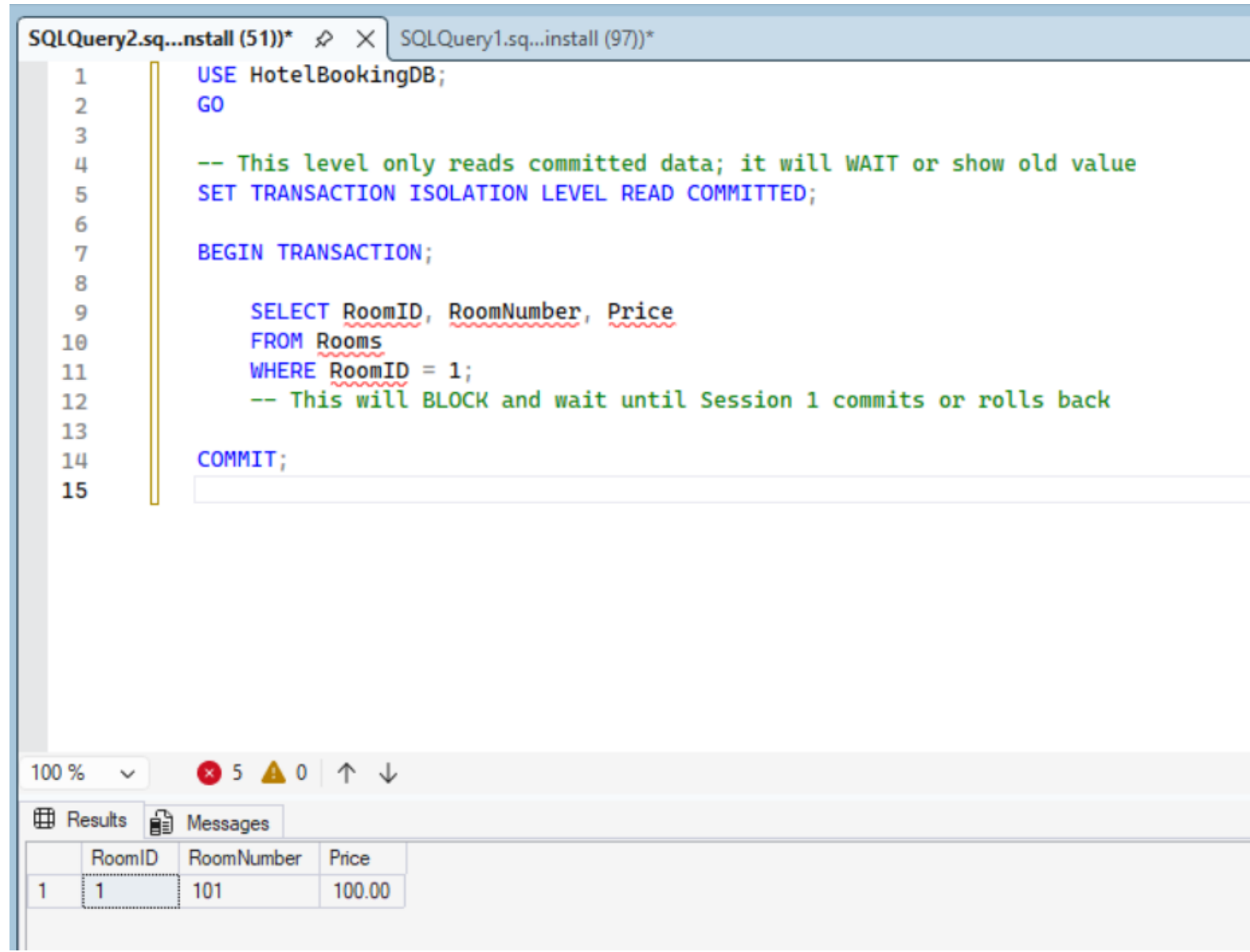
```
1  USE HotelBookingDB;
2  GO
3
4  BEGIN TRANSACTION;
5
6      -- Update room price WITHOUT committing
7      UPDATE Rooms
8      SET Price = 999.99
9      WHERE RoomID = 1;
10
11      PRINT 'Session 1: Price updated to 999.99 - NOT committed yet. Keep this window open.';
12
13      -- !! Do NOT commit or rollback yet. Switch to Session 2 now. !!
14
15      ROLLBACK;
16      PRINT 'Session 1: Transaction rolled back. Price is back to 100.00';
17
```

At the bottom, there is a 'Messages' pane. It shows the following message:

```
Session 1: Transaction rolled back. Price is back to 100.00
```

Below the message, it shows the completion time: 2026-02-27T17:51:38.9375977+05:30.

As we can see once the **session 1** released the page **session 2** captured and read the correct last committed value which was 100 as the **session 2** was in **read committed isolation level**.



The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are two tabs: "SQLQuery2.sq...nstall (51))*" and "SQLQuery1.sq...install (97))*". The active tab displays a SQL script with the following content:

```
1  USE HotelBookingDB;
2  GO
3
4  -- This level only reads committed data; it will WAIT or show old value
5  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
6
7  BEGIN TRANSACTION;
8
9      SELECT RoomID, RoomNumber, Price
10     FROM Rooms
11     WHERE RoomID = 1;
12     -- This will BLOCK and wait until Session 1 commits or rolls back
13
14 COMMIT;
15
```

Below the script, there is a status bar showing "100 %", "5" errors, "0" warnings, and navigation arrows. At the bottom, there are tabs for "Results" and "Messages". The "Results" tab is active, displaying a table with the following data:

	RoomID	RoomNumber	Price
1	1	101	100.00

This complete our experiment with **read
uncommitted** and **read committed** isolation level

Task B: Non-Repeatable Read

In Session 1, start a transaction and read the booking status of a specific room.

In Session 2, start a new transaction to update the booking status for the same room and commit.

Back in Session 1, try reading the booking status again and observe if it has changed.

Experiment with READ COMMITTED and REPEATABLE READ isolation levels.

Before moving to the **repeatable read** isolation level, we'll first observe why do we need it in the first place, to make sure that we really need it we'll first try the transaction with **read committed** isolation level

As you can see in the given below image, we have initialized the **session 1** transaction with **read committed** isolation level, and we are reading the booking status of the **booking id 3** and as you can see its showing **“Cancelled”**

SQLQuery2.sq...install (51))*

SQLQuery1.sq...nstall (97))*

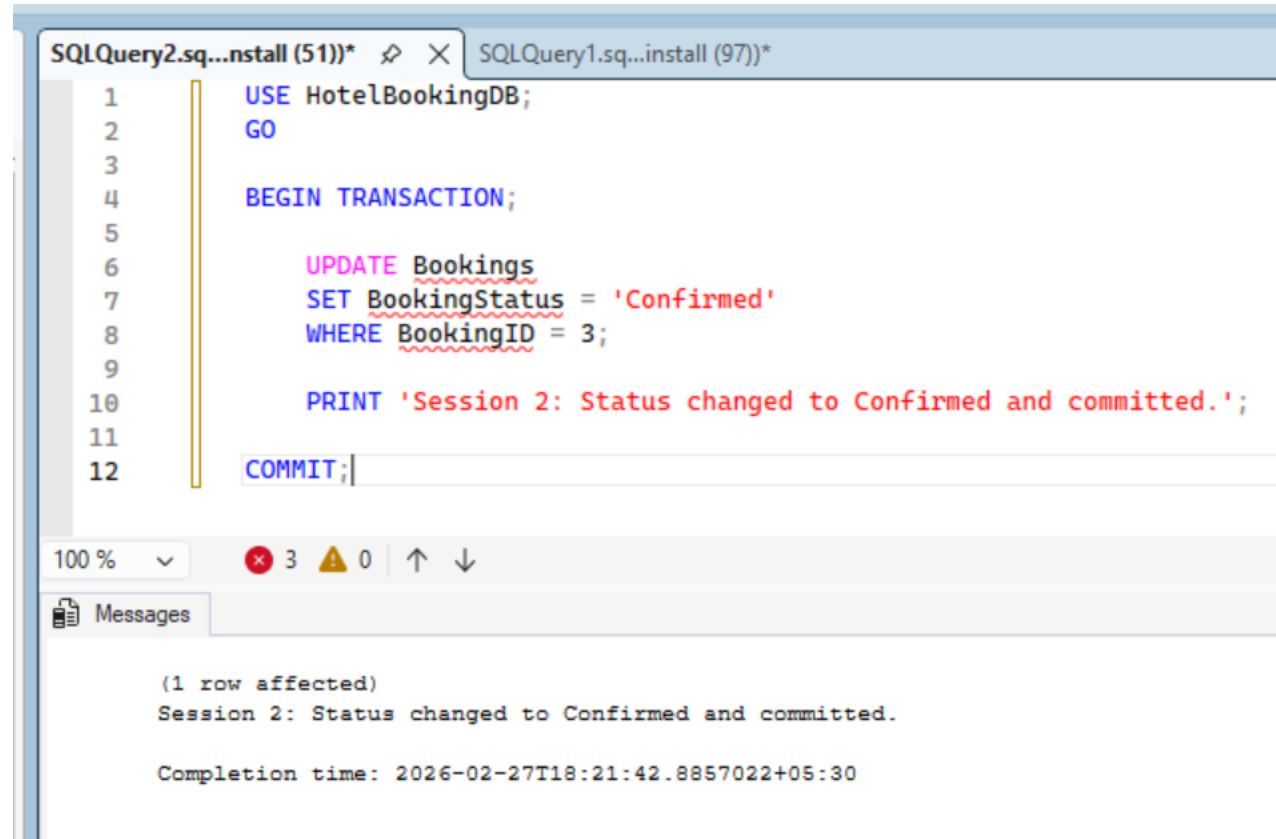
```
1  USE HotelBookingDB;
2  GO
3
4  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
5
6  BEGIN TRANSACTION;
7
8      -- First read of booking status
9      SELECT BookingID, RoomID, CustomerName, BookingStatus
10     FROM Bookings
11     WHERE BookingID = 3;
12     -- Shows: Cancelled
13
14     PRINT 'Session 1: First read done. Now run Session 2 script.';
15
16     -- Keep transaction open! Switch to Session 2.
17
```

100 % 6 0

Results Messages

	BookingID	RoomID	CustomerName	BookingStatus
1	3	3	Carol Williams	Cancelled

Now coming to **session 2** and updating the **booking status** from “Cancelled” to “Confirmed” of row with **BookingID 3** and committing the changes to the database



The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are two tabs: 'SQLQuery2.sql...install (51))*' and 'SQLQuery1.sql...install (97))*'. The active tab is 'SQLQuery2.sql...install (51))*'. The query window contains the following SQL code:

```
1  USE HotelBookingDB;
2  GO
3
4  BEGIN TRANSACTION;
5
6  UPDATE Bookings
7  SET BookingStatus = 'Confirmed'
8  WHERE BookingID = 3;
9
10 PRINT 'Session 2: Status changed to Confirmed and committed.';
11
12 COMMIT;
```

Below the query window, there is a status bar showing '100 %' zoom, '3' errors, and '0' warnings. Below the status bar is a 'Messages' tab. The messages window displays the following output:

```
(1 row affected)
Session 2: Status changed to Confirmed and committed.

Completion time: 2026-02-27T18:21:42.8857022+05:30
```

Now when the **session 1** reads the again the same row the BookingStatus changes from “Cancelled” to “Confirmed” because the **session 2** updated it.

This reads to the problem of **non-repeatable reads**. And now to overcome this issue we use the **Repeatable read isolation level**.

SQLQuery2.sq...install (51))* SQLQuery1.sq...nstall (97))*

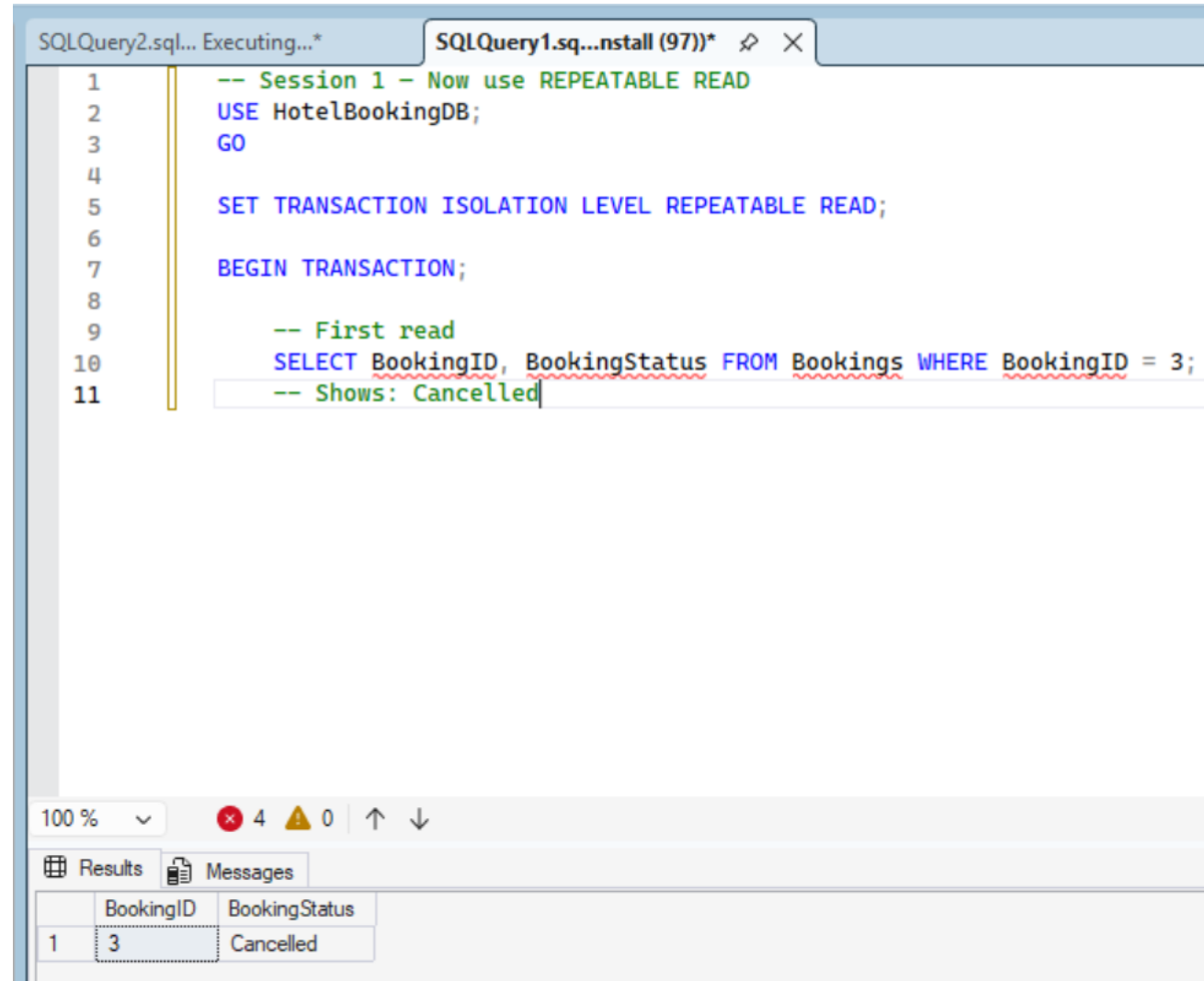
```
1  USE HotelBookingDB;
2  GO
3
4  SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
5
6  BEGIN TRANSACTION;
7
8  -- First read of booking status
9  SELECT BookingID, RoomID, CustomerName, BookingStatus
10 FROM Bookings
11 WHERE BookingID = 3;
12 -- Shows: Cancelled
13
14 PRINT 'Session 1: First read done. Now run Session 2 script.';
15
16 -- Keep transaction open! Switch to Session 2.
17 SELECT BookingID, RoomID, CustomerName, BookingStatus
18 FROM Bookings
19 WHERE BookingID = 3;
20 -- With READ COMMITTED: now shows 'Confirmed' - NON-REPEATABLE READ occurred!
21
22 COMMIT;
23
```

100 % 12 0 ↑ ↓

Results Messages

	BookingID	RoomID	CustomerName	BookingStatus
1	3	3	Carol Williams	Confirmed

Here we'll use the repeatable read isolation level to make sure that there is no problem of non repeatable read, in session 1 we read the row with bookingid = 3.



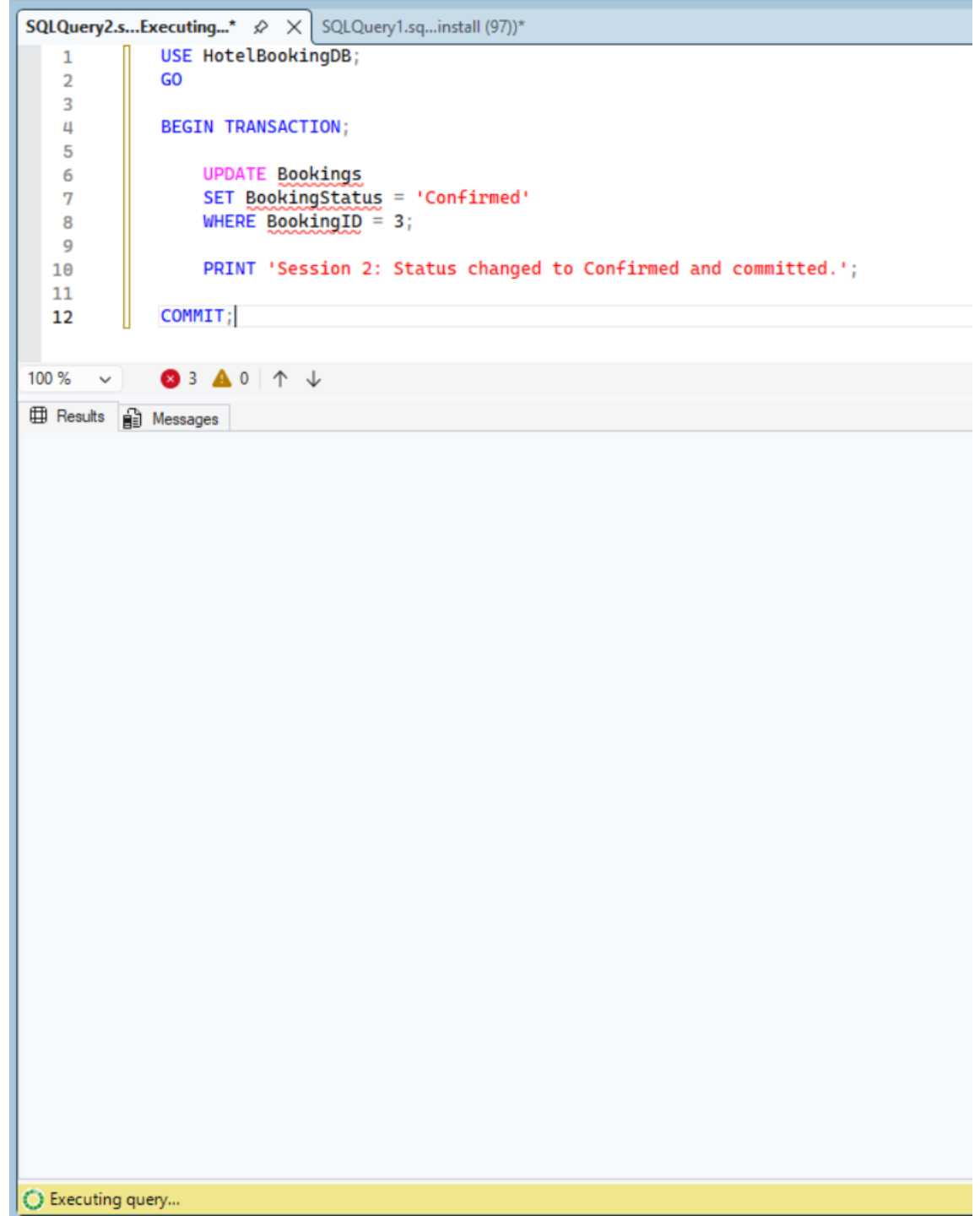
The screenshot shows a SQL Server Enterprise Manager interface. The top pane displays a query window with the following T-SQL code:

```
1  -- Session 1 - Now use REPEATABLE READ
2  USE HotelBookingDB;
3  GO
4
5  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
6
7  BEGIN TRANSACTION;
8
9  -- First read
10 SELECT BookingID, BookingStatus FROM Bookings WHERE BookingID = 3;
11 -- Shows: Cancelled
```

The bottom pane shows the 'Results' tab with a grid containing one row of data:

	BookingID	BookingStatus
1	3	Cancelled

In session 2 we try to update the bookingstatus to “Confirmed” of the bookingid = 3 but this query is getting blocked by the session 1 uncommitted transaction which is using the repeatable read isolation level

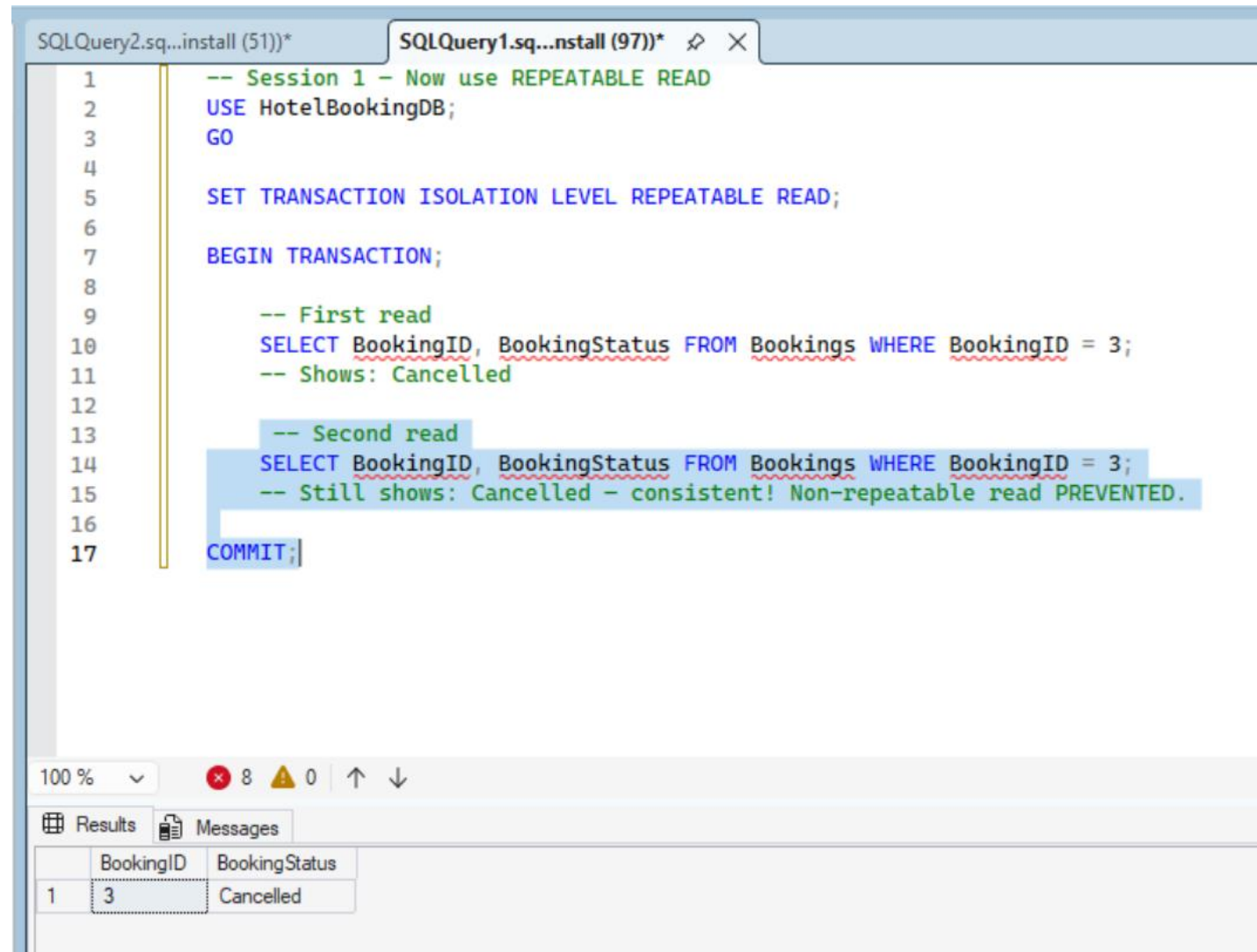


The screenshot shows a SQL Server Enterprise Manager interface. The top pane displays a query window titled "SQLQuery2.s...Executing...*" with the following SQL code:

```
1  USE HotelBookingDB;
2  GO
3
4  BEGIN TRANSACTION;
5
6      UPDATE Bookings
7      SET BookingStatus = 'Confirmed'
8      WHERE BookingID = 3;
9
10     PRINT 'Session 2: Status changed to Confirmed and committed.';
11
12 COMMIT;
```

The bottom pane shows the "Results" tab, which is currently empty. The status bar at the bottom indicates "Executing query...".

Now coming back to the session 1 and reading the row with bookingid = 3 reproduces the same result it did prior hence proving the repeatable read problem is evaded.



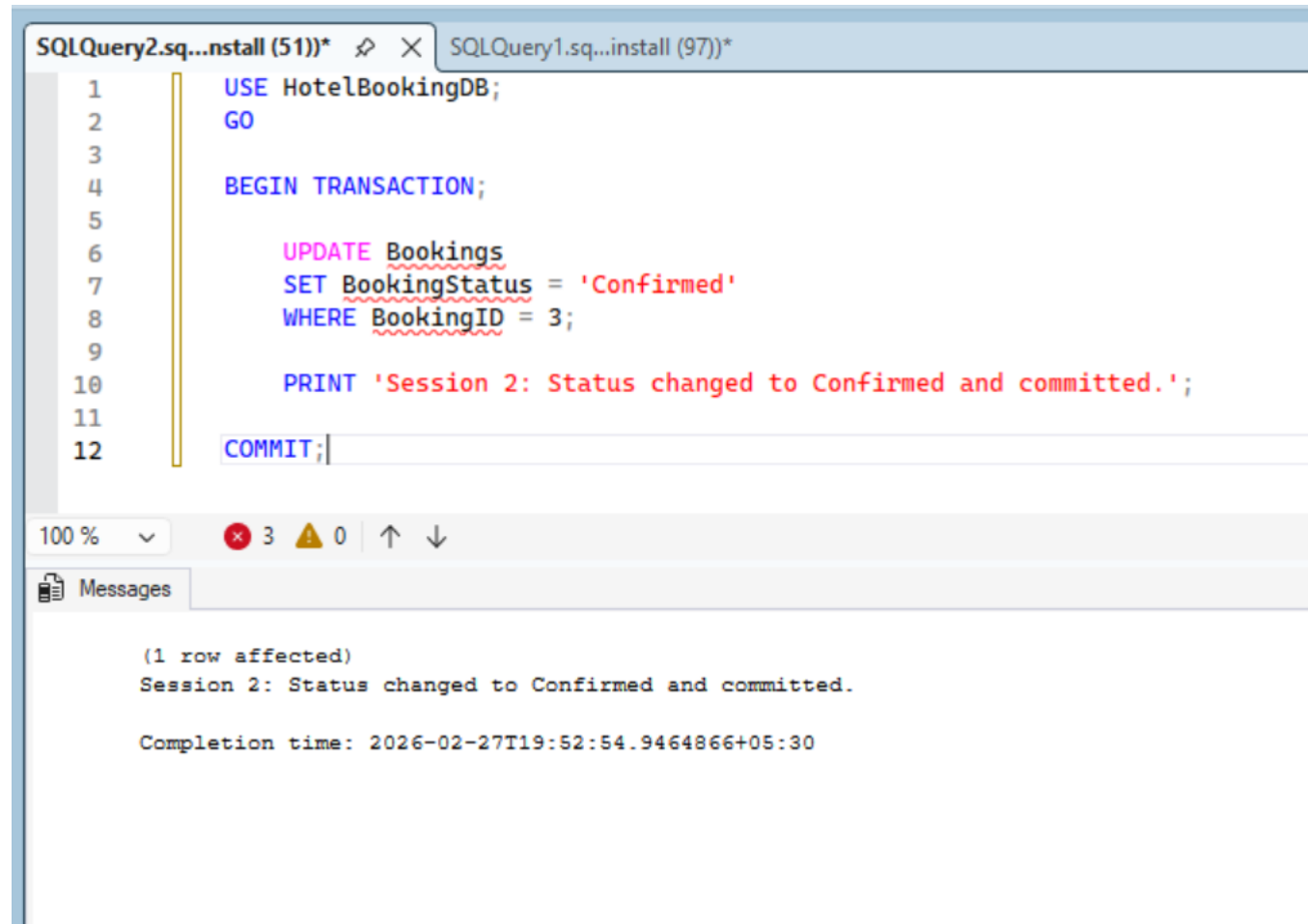
```
SQLQuery2.sql...install (51))*  SQLQuery1.sql...ninstall (97))*
1  -- Session 1 - Now use REPEATABLE READ
2  USE HotelBookingDB;
3  GO
4
5  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
6
7  BEGIN TRANSACTION;
8
9  -- First read
10 SELECT BookingID, BookingStatus FROM Bookings WHERE BookingID = 3;
11 -- Shows: Cancelled
12
13 -- Second read
14 SELECT BookingID, BookingStatus FROM Bookings WHERE BookingID = 3;
15 -- Still shows: Cancelled - consistent! Non-repeatable read PREVENTED.
16
17 COMMIT;
```

100 % 8 0

Results Messages

	BookingID	BookingStatus
1	3	Cancelled

After the session 1 is has completed its transaction and released its locks then the session 2 updates the row



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a query window with the following SQL code:

```
1  USE HotelBookingDB;  
2  GO  
3  
4  BEGIN TRANSACTION;  
5  
6  UPDATE Bookings  
7  SET BookingStatus = 'Confirmed'  
8  WHERE BookingID = 3;  
9  
10 PRINT 'Session 2: Status changed to Confirmed and committed.';  
11  
12 COMMIT;
```

The bottom pane shows the execution results:

```
(1 row affected)  
Session 2: Status changed to Confirmed and committed.  
  
Completion time: 2026-02-27T19:52:54.9464866+05:30
```

Task C: Phantom Read

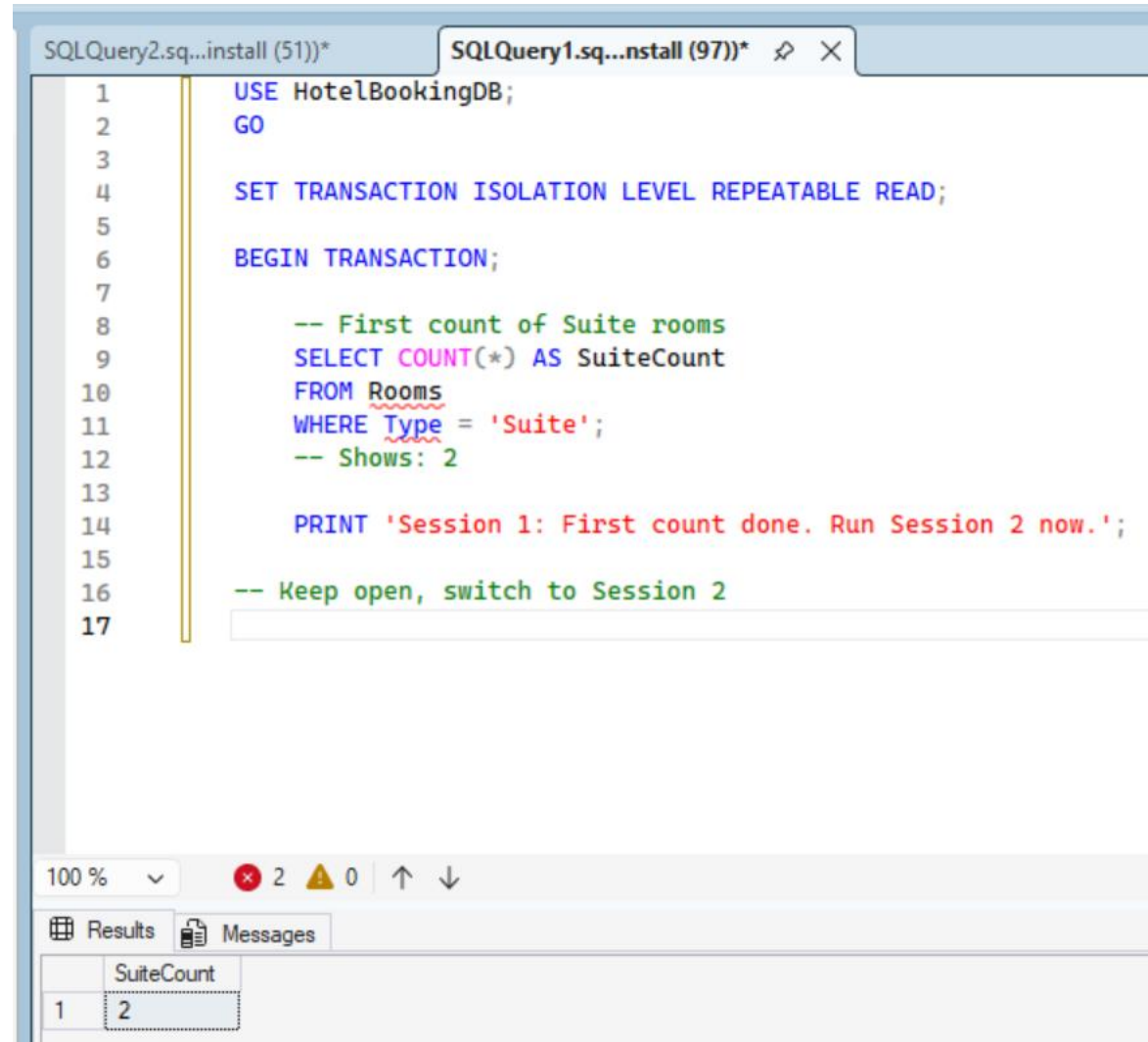
In Session 1, start a transaction and run a query to count all available rooms of type 'Suite'.

In Session 2, insert a new 'Suite' room into the Rooms table and commit.

Back in Session 1, run the same query again and check if the count remains consistent.

Experiment with REPEATABLE READ and SERIALIZABLE isolation levels.

In session 1 we start a transaction with repeatable read isolation level to show why we need the serializable isolation level in this transaction we are printing the count of rows with type = 'suite' right now its showing count as 2



The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are two tabs: 'SQLQuery2.sql...install (51))*' and 'SQLQuery1.sql...nstall (97))*'. The active tab is 'SQLQuery1.sql...nstall (97))*'. The main area displays a T-SQL script with line numbers 1 through 17. The script sets the transaction isolation level to REPEATABLE READ, begins a transaction, and queries the 'Rooms' table for 'Suite' rooms. The results pane at the bottom shows a single row with 'SuiteCount' of 2.

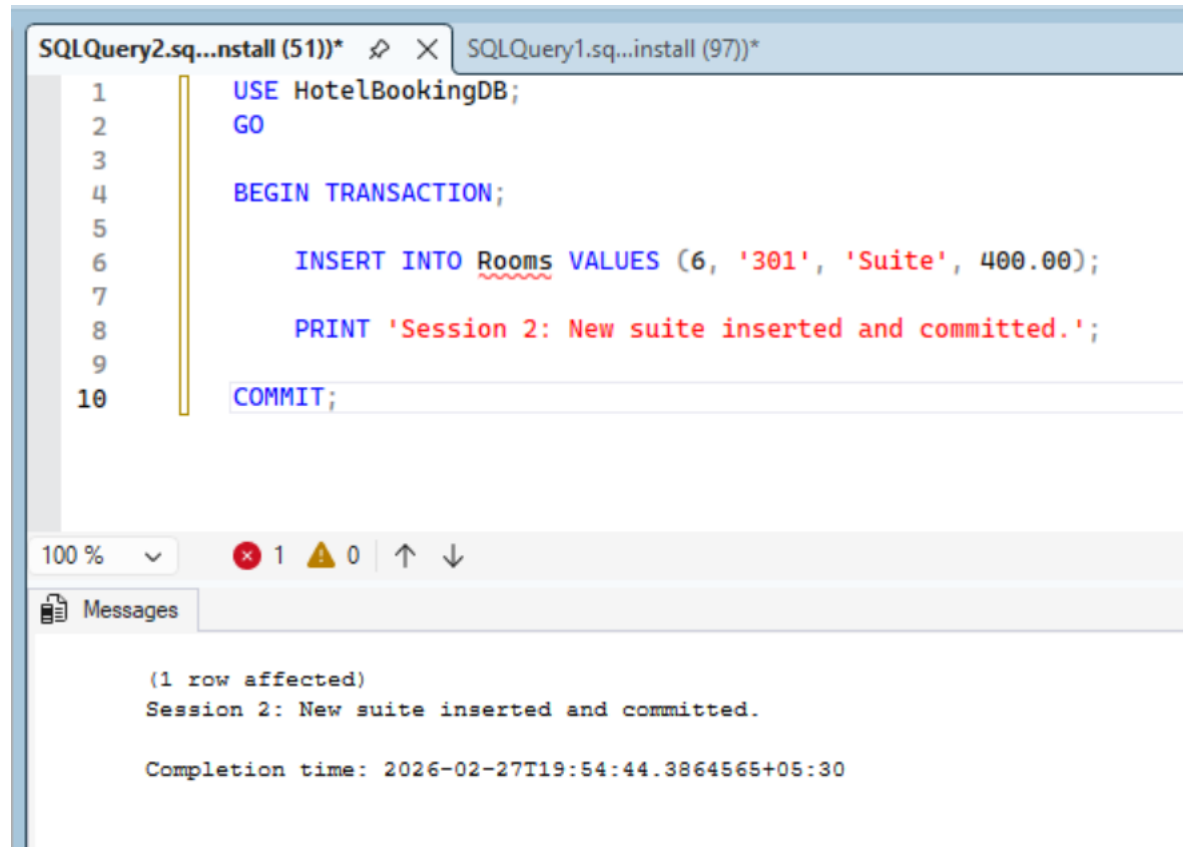
```
1  USE HotelBookingDB;
2  GO
3
4  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
5
6  BEGIN TRANSACTION;
7
8  -- First count of Suite rooms
9  SELECT COUNT(*) AS SuiteCount
10 FROM Rooms
11 WHERE Type = 'Suite';
12 -- Shows: 2
13
14 PRINT 'Session 1: First count done. Run Session 2 now.';
15
16 -- Keep open, switch to Session 2
17
```

100 % 2 0

Results Messages

	SuiteCount
1	2

In session 2 we insert a row with the **type = 'Suite'** in the table **Rooms** and committing so that the changes are stored in database



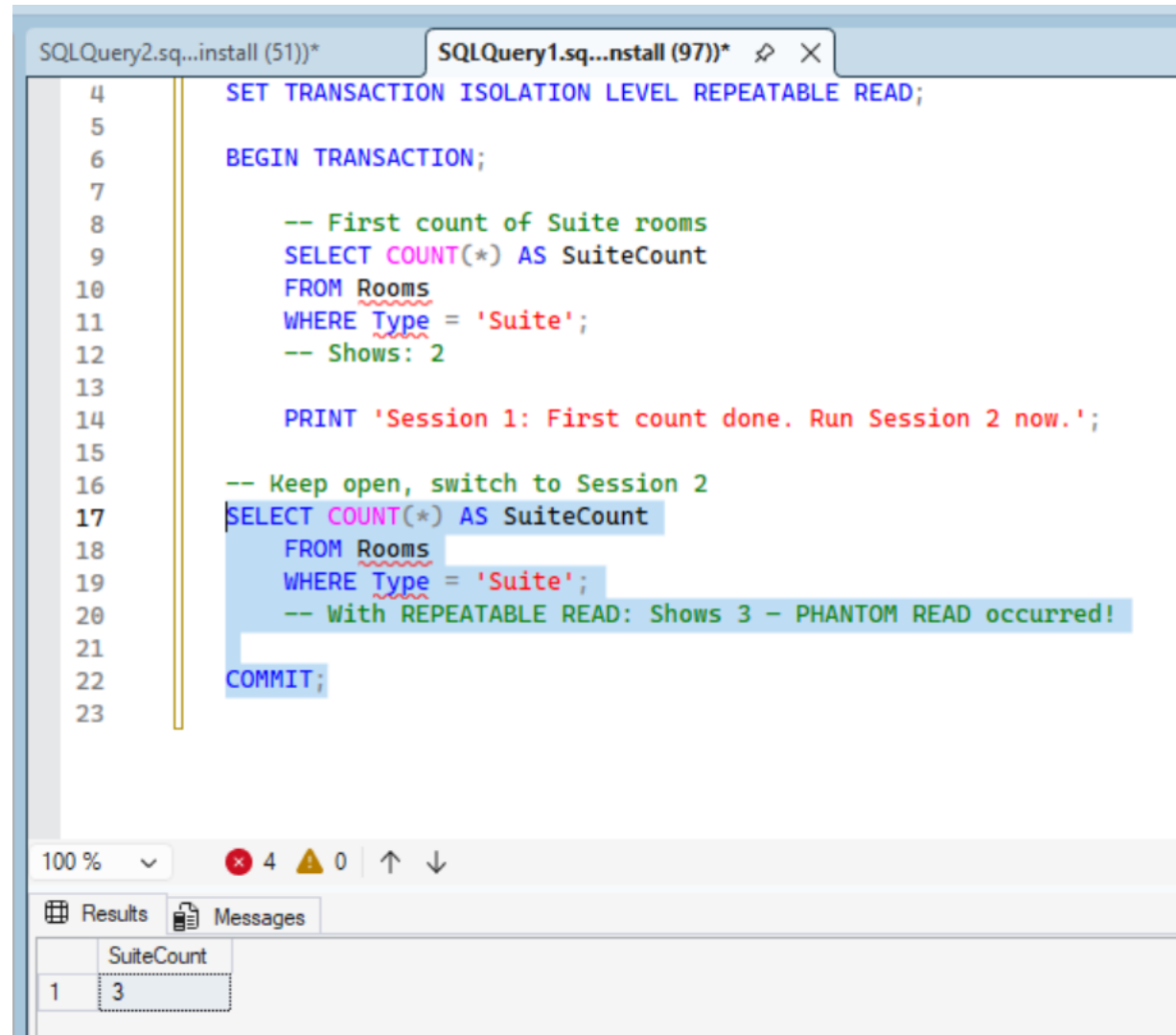
The screenshot displays the SQL Server Enterprise Manager interface. At the top, there are two tabs: 'SQLQuery2.sql...ninstall (51))*' and 'SQLQuery1.sql...install (97))*'. The main window shows a SQL script with the following lines:

```
1  USE HotelBookingDB;  
2  GO  
3  
4  BEGIN TRANSACTION;  
5  
6      INSERT INTO Rooms VALUES (6, '301', 'Suite', 400.00);  
7  
8      PRINT 'Session 2: New suite inserted and committed.';  
9  
10 COMMIT;
```

Below the script, there is a status bar showing '100 %' zoom, a red 'x' icon with '1', a yellow triangle icon with '0', and up/down arrow icons. At the bottom, there is a 'Messages' pane with the following output:

```
(1 row affected)  
Session 2: New suite inserted and committed.  
  
Completion time: 2026-02-27T19:54:44.3864565+05:30
```

In session 1 we are again displaying the count of rows with type = "Suite" and now its showing count as 3 which is different from the previous count i.e. 2 so this is problem of phantom read and to avoid this there is a isolation level named as serializable



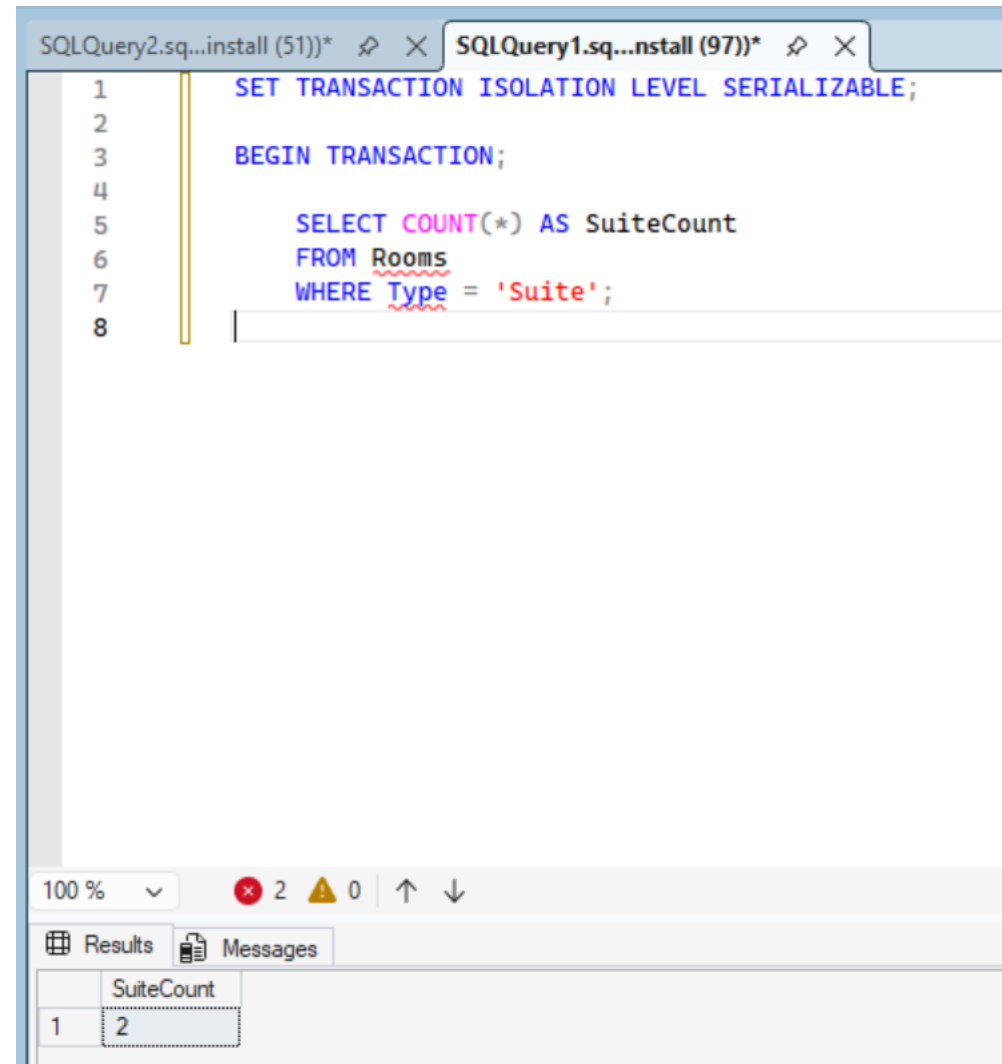
```
SQLQuery2.sq...install (51))*  SQLQuery1.sq...ninstall (97))* X
4      SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
5
6      BEGIN TRANSACTION;
7
8      -- First count of Suite rooms
9      SELECT COUNT(*) AS SuiteCount
10     FROM Rooms
11     WHERE Type = 'Suite';
12     -- Shows: 2
13
14     PRINT 'Session 1: First count done. Run Session 2 now.';
15
16     -- Keep open, switch to Session 2
17     SELECT COUNT(*) AS SuiteCount
18     FROM Rooms
19     WHERE Type = 'Suite';
20     -- With REPEATABLE READ: Shows 3 - PHANTOM READ occurred!
21
22     COMMIT;
23
```

100 % 4 0

Results Messages

	SuiteCount
1	3

In session 1 we begin a transaction with serializable isolation level and again take a count of rooms with type = “Suite” and this time the count is 2



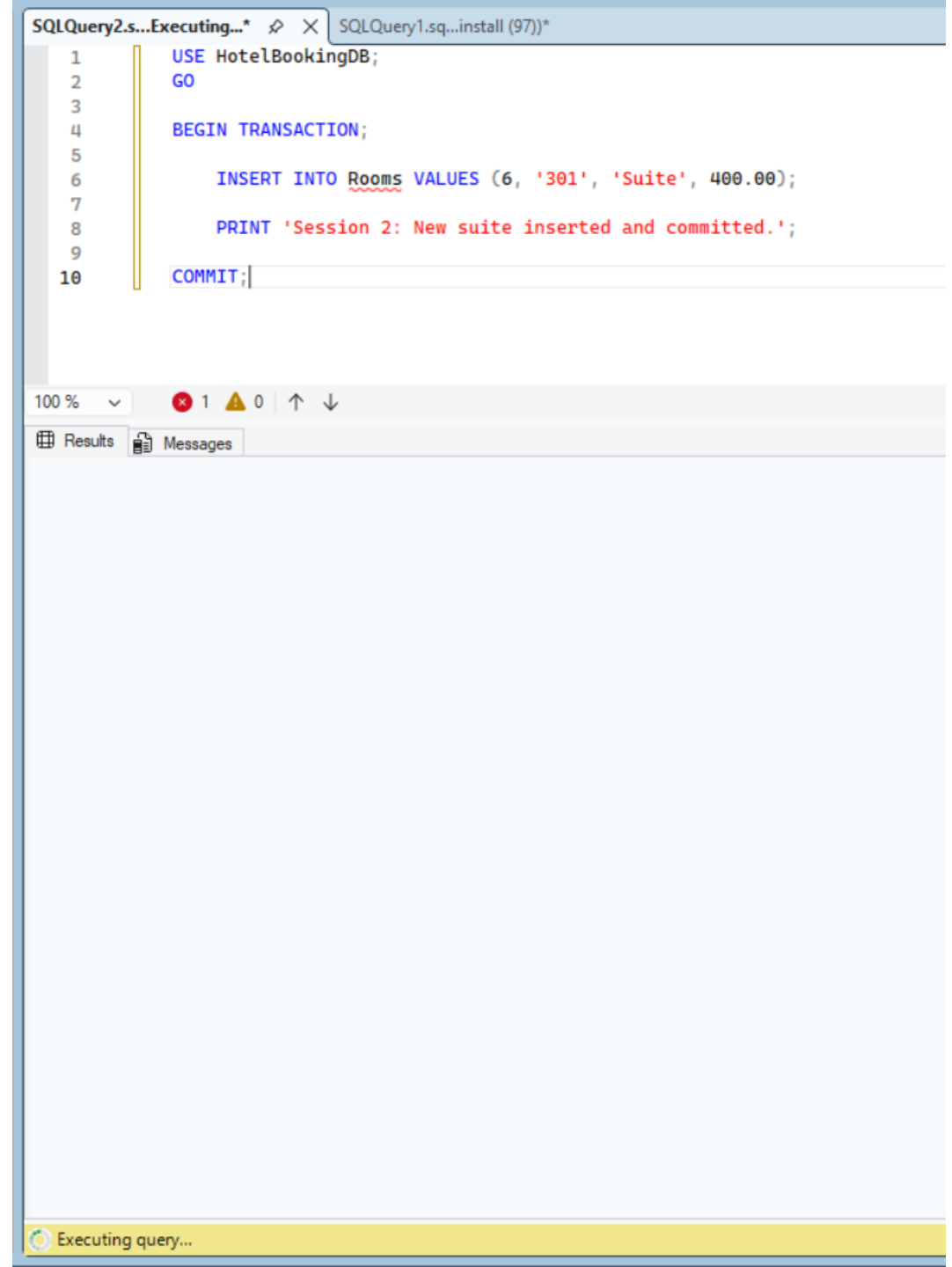
```
SQLQuery2.sq...install (51))*  SQLQuery1.sq...nstall (97))*  
1  SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
2  
3  BEGIN TRANSACTION;  
4  
5      SELECT COUNT(*) AS SuiteCount  
6      FROM Rooms  
7      WHERE Type = 'Suite';  
8
```

100 % 2 0

Results Messages

	SuiteCount
1	2

In session 2 we execute a insert command that inserts the row in Rooms table and this time its put on hold by the session 1 transaction

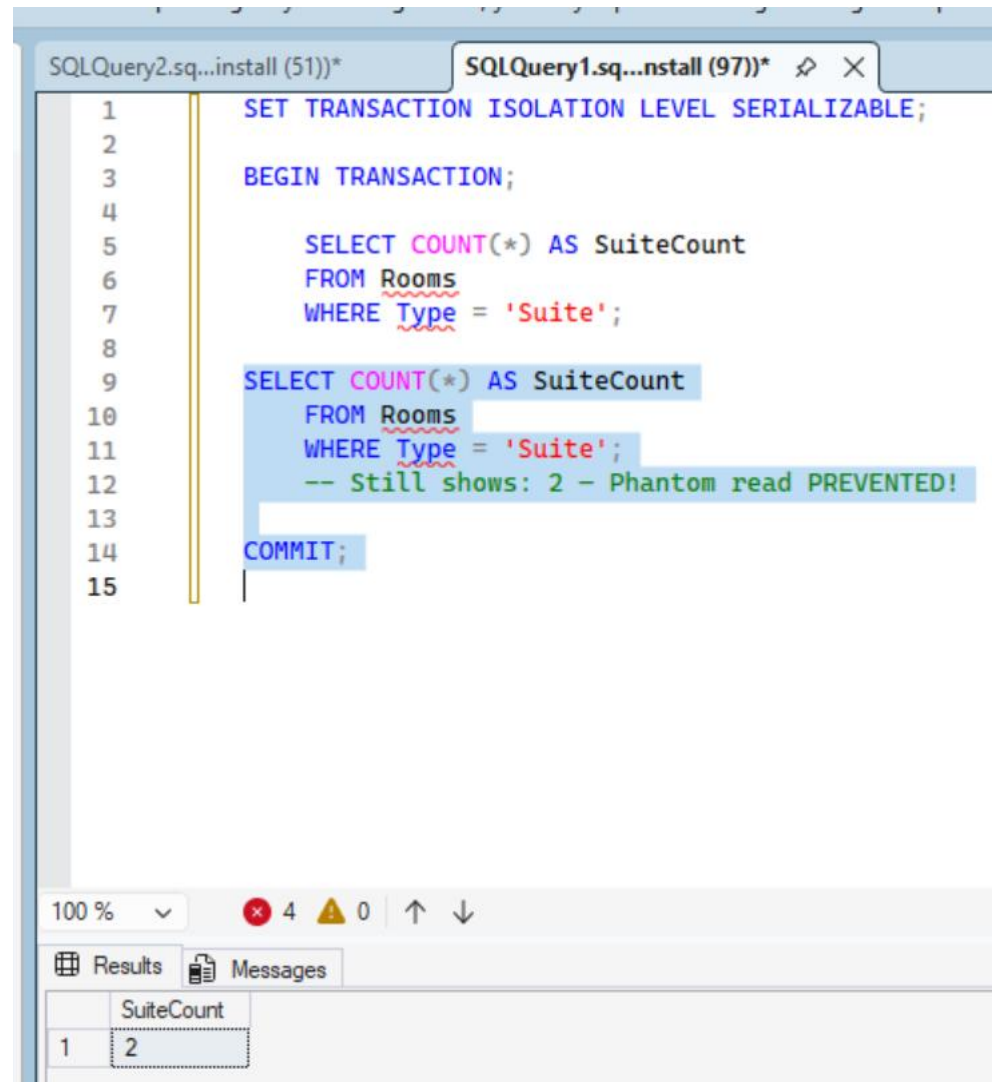


The screenshot displays the SQL Server Enterprise Manager interface. At the top, there are two tabs: 'SQLQuery2.s...Executing...*' and 'SQLQuery1.sq...install (97))*'. The active tab shows a SQL script with the following lines:

```
1  USE HotelBookingDB;
2  GO
3
4  BEGIN TRANSACTION;
5
6      INSERT INTO Rooms VALUES (6, '301', 'Suite', 400.00);
7
8      PRINT 'Session 2: New suite inserted and committed.';
9
10 COMMIT;
```

Below the script editor, there is a status bar showing '100 %' zoom, a red 'x' icon with the number '1', a yellow triangle icon with the number '0', and up/down arrow icons. Below this is a tabbed interface with 'Results' and 'Messages' tabs. The 'Results' tab is currently selected and is empty. At the bottom of the window, a yellow status bar indicates 'Executing query...'.

In session 1 we again take a count of rooms with type = "Suite" and this time the count is still 2 and the transaction of session 1 commits



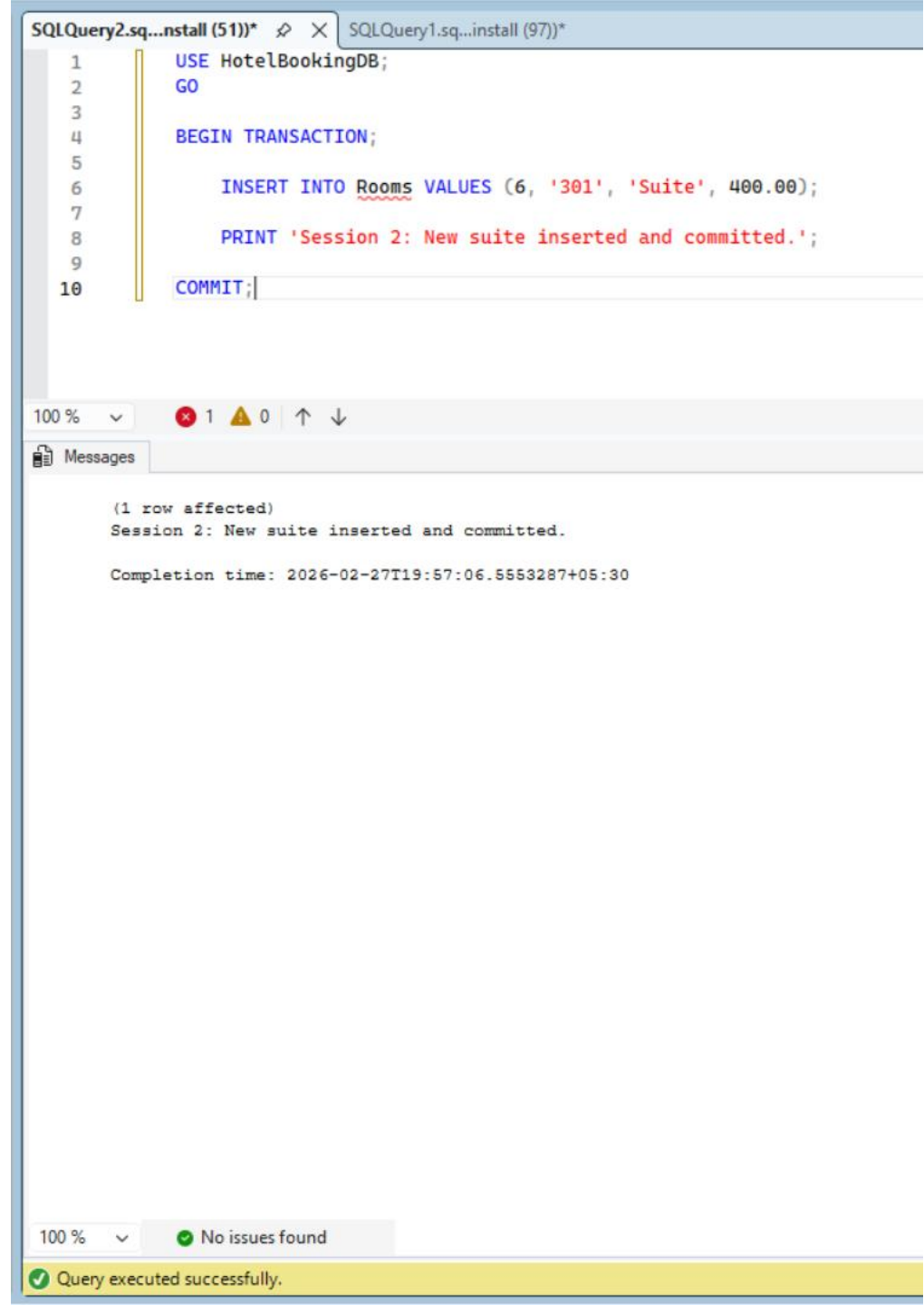
The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are two tabs: "SQLQuery2.sql...install (51))*" and "SQLQuery1.sql...ninstall (97))*". The active tab is "SQLQuery1.sql...ninstall (97))*". The main area displays a SQL script with line numbers 1 through 15. The script is as follows:

```
1 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
2
3 BEGIN TRANSACTION;
4
5     SELECT COUNT(*) AS SuiteCount
6     FROM Rooms
7     WHERE Type = 'Suite';
8
9     SELECT COUNT(*) AS SuiteCount
10    FROM Rooms
11    WHERE Type = 'Suite';
12    -- Still shows: 2 - Phantom read PREVENTED!
13
14 COMMIT;
```

At the bottom of the window, there is a status bar showing "100 %", "4" errors, "0" warnings, and "0" messages. Below the status bar are two tabs: "Results" and "Messages". The "Results" tab is active, showing a grid with one row and one column. The column is labeled "SuiteCount" and the row contains the value "2".

	SuiteCount
1	2

After the completion of the session 1 transaction now the session 2 is able to insert the row into the Rooms table avoid the phantom read problem by using the serializable isolation level



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL query script for 'SQLQuery2.sql...n stall (51))'. The query is as follows:

```
1  USE HotelBookingDB;
2  GO
3
4  BEGIN TRANSACTION;
5
6  INSERT INTO Rooms VALUES (6, '301', 'Suite', 400.00);
7
8  PRINT 'Session 2: New suite inserted and committed.';
9
10 COMMIT;
```

The bottom pane shows the 'Messages' tab with the following output:

```
(1 row affected)
Session 2: New suite inserted and committed.

Completion time: 2026-02-27T19:57:06.5553287+05:30
```

At the bottom of the interface, a status bar indicates '100 %' zoom, 'No issues found', and a green checkmark with the message 'Query executed successfully.'